ROUND 6 – ADVANCED+ (C++)

1. Balanced Parentheses Check

Buggy Code:

```
#include <iostream>
using namespace std;

int main() {
    string s = "(()";
    int count = 0;
    for(char c : s){
        if(c == '(')
            count++;
        else
            count--;
    }
    if(count == 0)
        cout << "Balanced";
    else
        cout << "Not Balanced";
    return 0;
}
```

Correct Code:

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    string s = "(()())";
    stack<char> st;
    bool balanced = true;

    for(char c : s){
        if(c == '(')
            st.push(c);
        else{
            if(st.empty()){
                balanced = false;
                break;
            }
            st.pop();
        }
    }

    if(!st.empty())
        balanced = false;

    cout << (balanced ? "Balanced" : "Not Balanced");
    return 0;
}
```

2. All Permutations of a String

Buggy Code:

```
#include <iostream>
using namespace std;

int main() {
    string s = "ABC";
    cout << s;   // incorrect
```

```
    return 0;
}
```

Correct Code:

```
#include <iostream>
using namespace std;

void permute(string s, int l, int r){
    if(l == r){
        cout << s << endl;
        return;
    }
    for(int i = l; i <= r; i++){
        swap(s[l], s[i]);
        permute(s, l + 1, r);
    }
}

int main() {
    string s = "ABC";
    permute(s, 0, s.length() - 1);
    return 0;
}
```

3. Longest Palindromic Substring

Buggy Code:

```
#include <iostream>
using namespace std;

int main() {
    string s = "babad";
    cout << s;
    return 0;
}
```

Correct Code:

```
#include <iostream>
using namespace std;

int expand(string s, int left, int right){
    while(left >= 0 && right < s.length() && s[left] == s[right]){
        left--;
        right++;
    }
    return right - left - 1;
}

int main() {
    string s = "babad";
    int start = 0, end = 0;

    for(int i = 0; i < s.length(); i++){
        int len1 = expand(s, i, i);
        int len2 = expand(s, i, i + 1);
        int len = max(len1, len2);

        if(len > end - start){
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }

    cout << s.substr(start, end - start + 1);
    return 0;
}
```

4. Stack Using Array

Buggy Code:

```
#include <iostream>
using namespace std;

int stackArr[5], top = 0;

int main() {
    stackArr[top++] = 10;
    stackArr[top++] = 20;
    cout << stackArr[top];
    return 0;
}
```

Correct Code:

```
#include <iostream>
using namespace std;

int stackArr[5];
int top = -1;

void push(int x){
    if(top == 4){
        cout << "Overflow" << endl;
        return;
    }
    stackArr[++top] = x;
}

void pop(){
    if(top == -1){
        cout << "Underflow" << endl;
        return;
    }
    top--;
}

int main() {
    push(10);
    push(20);
    push(30);

    for(int i = top; i >= 0; i--)
        cout << stackArr[i] << " ";
    return 0;
}
```

5. Queue Using Array

Buggy Code:

```
#include <iostream>
using namespace std;

int q[5], front = 0, rear = 0;

int main() {
    q[rear++] = 10;
    q[rear++] = 20;
    cout << q[front];
    return 0;
}
```

Correct Code:

```
#include <iostream>
using namespace std;

int q[5];
int front = -1, rear = -1;

void enqueue(int x){
    if(rear == 4){
        cout << "Queue Full" << endl;
        return;
    }
    if(front == -1) front = 0;
    q[++rear] = x;
}

void dequeue(){
    if(front == -1 || front > rear){
        cout << "Queue Empty" << endl;
        return;
    }
    front++;
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);

    for(int i = front; i <= rear; i++)
        cout << q[i] << " ";
    return 0;
}
```

6. Majority Element in Array

Buggy Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {2,2,1,1};
    cout << arr[0];
    return 0;
}
```

Correct Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {2,2,1,1,2,2,2};
    int n = 7;
    int count = 0, candidate = -1;

    for(int i = 0; i < n; i++){
        if(count == 0){
            candidate = arr[i];
            count = 1;
        } else if(arr[i] == candidate){
            count++;
        } else {
            count--;
        }
    }
    count = 0;
```

```
    for(int i = 0; i < n; i++)
        if(arr[i] == candidate)
            count++;

    if(count > n/2)
        cout << "Majority Element: " << candidate;
    else
        cout << "No Majority Element";

    return 0;
}
```

7. Remove Spaces Without replace()

Buggy Code:

```
#include <iostream>
using namespace std;

int main() {
    string s = "hello world";
    cout << s;
    return 0;
}
```

Correct Code:

```
#include <iostream>
using namespace std;

int main() {
    string s = "hello world";
    string result = "";

    for(int i = 0; i < s.length(); i++){
        if(s[i] != ' ')
            result += s[i];
    }

    cout << result;
    return 0;
}
```

8. Maximum Sum Subarray (Kadane)

Buggy Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {-2,1,-3,4};
    int sum = 0;
    for(int i = 0; i < 4; i++)
        sum += arr[i];
    cout << sum;
    return 0;
}
```

Correct Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {-2,1,-3,4,-1,2,1,-5,4};
    int n = 9;
```

```
    int maxSum = arr[0], currSum = arr[0];

    for(int i = 1; i < n; i++){
        currSum = max(arr[i], currSum + arr[i]);
        maxSum = max(maxSum, currSum);
    }

    cout << "Maximum Sum: " << maxSum;
    return 0;
}
```

9. Check String Rotation

Buggy Code:

```
#include <iostream>
using namespace std;

int main() {
    string s1 = "abcd";
    string s2 = "cdab";
    if(s1 == s2)
        cout << "Rotation";
    return 0;
}
```

Correct Code:

```
#include <iostream>
using namespace std;

int main() {
    string s1 = "abcd";
    string s2 = "cdab";

    if(s1.length() != s2.length()){
        cout << "Not Rotation";
        return 0;
    }

    string temp = s1 + s1;

    if(temp.find(s2) != string::npos)
        cout << "Rotation";
    else
        cout << "Not Rotation";

    return 0;
}
```

10. Longest Consecutive Sequence in Array

Buggy Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {100,4,200,1,3,2};
    cout << arr[0];
    return 0;
}
```

Correct Code:

```
#include <iostream>
#include <set>
```

```
using namespace std;

int main() {
    int arr[] = {100,4,200,1,3,2};
    int n = 6;
    set<int> s(arr, arr + n);
    int longest = 0;

    for(int x : s){
        if(s.find(x - 1) == s.end()){
            int current = x;
            int count = 1;
            while(s.find(current + 1) != s.end()){
                current++;
                count++;
            }
            longest = max(longest, count);
        }
    }

    cout << "Longest Consecutive Length: " << longest;
    return 0;
}
```